Disciplina: Técnicas Avançadas de Banco de Dados Relacional e Não Relacional

Store Procedures – Usando Tabela

Profª. Drª. Giovana Angélica Ros
giovana.ros@cps.sp.gov.br

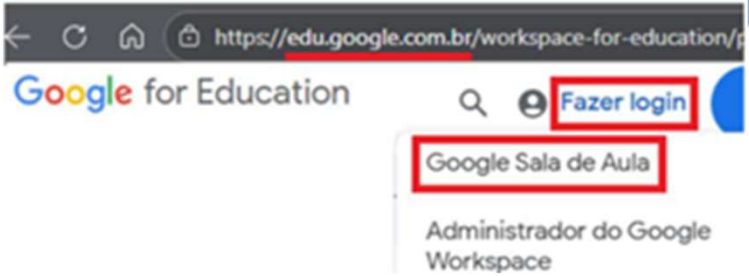
Fatec
Presidente Prudente

Centro Paula Souza

Acessar

Ferramenta de aula Classroom (Google sala de aula)

- Acessar pelo navegador https://edu.google.com.br/
- Informar seu usuário do **Gmail** (e-mail ou celular)
- Informar a sua senha do **Gmail**
- **Participar da turma** (sinal de **+** canto superior direito)



- Turma: **T A Banco de Dados**
- Código da turma: **nm5psmsa**

Padrões de Nomeação das Store Procedures

- Padrões para manter o banco de dados organizado, legível e fácil de manter
  - muitos desenvolvedores preferem **pr_** para evitar qualquer confusão futura, por exemplo, com SqlServer, que usa sp_ para procedures internas
  - O nome deve descrever exatamente o que a procedure faz, seguindo a lógica de **Verbo + Substantivo**, exemplo: sp_atualizar_estoque()
- Estilo de Escrita (Casing)
  - **Snake Case (Recomendado para MySQL):** Tudo em minúsculo com underlines. Exemplo: sp_calcular_folha_pagamento()
  - **Pascal Case:** Cada palavra começa com maiúscula. Exemplo: Exemplo: spGerarRelatorioVendas()

Store Procedure – Usando Tabela - Select

```
CREATE TABLE produtos (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nome VARCHAR(100),
   estoque INT,
   preco DECIMAL(10, 2)
);

INSERT INTO produtos (nome, estoque, preco) VALUES
('Teclado Slim', 50, 250.00),
('Mouse Gamer', 10, 150.00);
```

- Lista os produtos que estão com estoque abaixo de um limite a definir.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS pr_verificar_estoque $$
CREATE PROCEDURE pr_verificar_estoque(IN limite INT)
BEGIN
   SELECT nome, estoque
   FROM produtos
   WHERE estoque < limite;
END $$
DELIMITER ;

CALL pr_verificar_estoque(15);
```

nome	estoque
Mouse Gamer	10

Store Procedure - Select com IF ELSE

- Esta procedure realiza uma venda, diminui o estoque de um produto, se houver saldo suficiente.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS pr_vender_produto $$
CREATE PROCEDURE pr_vender_produto(IN p_id INT, IN qtd_venda INT)
BEGIN
   DECLARE v_estoque_atual INT;  -- Pega o estoque atual do produto
   SELECT estoque INTO v_estoque_atual FROM produtos WHERE id = p_id;
   -- Verifica se tem estoque suficiente
   IF v_estoque_atual >= qtd_venda THEN
      UPDATE produtos
      SET estoque = estoque - qtd_venda
      WHERE id = p_id;
      SELECT 'Venda realizada com sucesso!' AS Resultado;
   ELSE
      SELECT 'Erro: Estoque insuficiente.' AS Resultado;
   END IF;
END $$
DELIMITER ;

CALL pr_vender_produto(1,5);
```

SELECT de consulta à tabela

SELECT de apresentação de frase

Resultado
Venda realizada com sucesso!

Store Procedure – Select com cálculos

- Exemplo: Criar um **relatório financeiro mensal**, onde cruza os dados de duas tabelas e aplica cálculos de impostos e descontos.
- Calcula o faturamento bruto, aplica um imposto (ex: 10%) e mostra o lucro líquido estimado por produto em um determinado mês.

```
CREATE TABLE vendas (
   id INT AUTO_INCREMENT PRIMARY KEY,
   produto_id INT,
   quantidade INT,
   data_venda DATE,
   FOREIGN KEY (produto_id) REFERENCES produtos(id)
);

-- Inserindo algumas vendas para teste
INSERT INTO vendas (produto_id, quantidade, data_venda) VALUES
(1, 2, '2026-03-01'),
(1, 1, '2026-03-05'),
(2, 3, '2026-03-10');
```

Store Procedure – Select com cálculos

```
DELIMITER $$
DROP PROCEDURE IF EXISTS RelatorioFinanceiroMensal $$
CREATE PROCEDURE RelatorioFinanceiroMensal(IN p_mes INT, IN p_ano INT)
BEGIN
DECLARE v_taxa_imposto DECIMAL(4,2) DEFAULT 0.10; -- Definindo uma constante de imposto (10%)
   SELECT
      p.nome AS Produto,
      SUM(v.quantidade) AS Total_Vendido,
      SUM(v.quantidade * p.preco) AS Faturamento_Bruto,
      -- Cálculo: (Qtd * Preco) * Taxa
      SUM(v.quantidade * p.preco) * v_taxa_imposto AS Imposto_Devido,
      -- Cálculo Líquido: Bruto - Imposto
      SUM(v.quantidade * p.preco) * (1 - v_taxa_imposto) AS Faturamento_Liquido
   FROM produtos p
   INNER JOIN vendas v ON p.id = v.produto_id
   WHERE MONTH(v.data_venda) = p_mes AND YEAR(v.data_venda) = p_ano
   GROUP BY p.id;
END $$
DELIMITER ;

CALL RelatorioFinanceiroMensal(3,2026);
```

Produto	Total_Vendido	Faturamento_Bruto	Imposto_Devido	Faturamento_Liquido
Teclado Slim	3	750.00	75.0000	675.0000
Mouse Gamer	3	450.00	45.0000	405.0000

Exercícios para trabalho – vale 1 ponto

- **Exercício 1: Atualização de Status por Inatividade.** Crie uma procedure chamada pr_desativar_clientes_antigos. Ela deve receber uma data como parâmetro (p_data_limite). A procedure deve atualizar a coluna status para 'Inativo' de todos os clientes na tabela clientes cuja data_ultima_compra seja anterior à data informada.
- CALL pr_desativar_clientes_antigos("2023-05-11");

id	nome	status	data_ultima_compra
1	Ana Silva	Ativo	2023-05-10
2	Bruno Costa	Ativo	2025-12-20
3	Carla Souza	Ativo	2022-01-15

Após .....

id	nome	status	data_ultima_compra
1	Ana Silva	Inativo	2023-05-10
2	Bruno Costa	Ativo	2025-12-20
3	Carla Souza	Inativo	2022-01-15

Exercícios para trabalho – vale 1 ponto

- **Exercício 2: Cadastro com Validação de Segurança.** Crie uma procedure pr_adicionar_usuario que recebe p_nome, p_email e p_idade. A procedure deve inserir o usuário na tabela usuarios, mas **apenas se** a idade for maior ou igual a 18 anos. Se for menor, a procedure deve retornar uma mensagem de erro usando um SELECT 'Erro: Idade mínima não atingida’.

- CALL pr_adicionar_usuario "Ana","ana@gmail.com",20)

Exercícios para trabalho – vale 1 ponto

- **Exercício 3: Reajuste Salarial por Departamento.** Crie uma procedure pr_reajuste_salarial que recebe o nome de um departamento (p_depto) e um valor decimal de aumento (p_aumento). A procedure deve somar esse valor fixo ao salário de todos os funcionários da tabela funcionarios que pertencem ao departamento informado.

- CALL pr_reajuste_salarial('TI', 5);

id	nome	departamento	salario
1	Ricardo Melo	TI	5000.00
2	Fernanda Lima	TI	5500.00
3	Roberto Junior	RH	3000.00
4	Juliana Paes	Vendas	3500.00

Após.....

id	nome	departamento	salario
1	Ricardo Melo	TI	5005.00
2	Fernanda Lima	TI	5505.00
3	Roberto Junior	RH	3000.00
4	Juliana Paes	Vendas	3500.00

Exercícios para trabalho – vale 1 ponto

- **Exercício 4: Relatório de Vendas com Parâmetro de Saída (OUT).** Crie uma procedure pr_total_vendas_periodo que receba duas datas (início e fim) e um parâmetro de saída chamado p_total_final. A procedure deve calcular a soma de todos os valores da tabela vendas dentro desse intervalo e armazenar o resultado no parâmetro de saída.
- CALL pr_total_vendas_periodo('2026-01-10', '2026-03-12', @resultado);
- SELECT @resultado;

id	produto_id	valor_total	data_venda
1	1	4500.00	2026-01-15
2	2	240.00	2026-02-10
3	1	4500.00	2026-03-05
4	3	900.00	2026-03-08

@resultado
10140.00

Exercícios para trabalho – vale 1 ponto

- **Exercício 5: Controle de Estoque com Verificação Existencial.** Crie uma procedure pr_remover_produto. Ela recebe o p_id de um produto. Antes de deletar, a procedure deve verificar se o produto existe na tabela produtos. Se existir, deleta e retorna "Sucesso". Se não existir, retorna "Produto não encontrado".

- CALL pr_remover_produto(9);

Resultado
Erro: Produto não encontrado